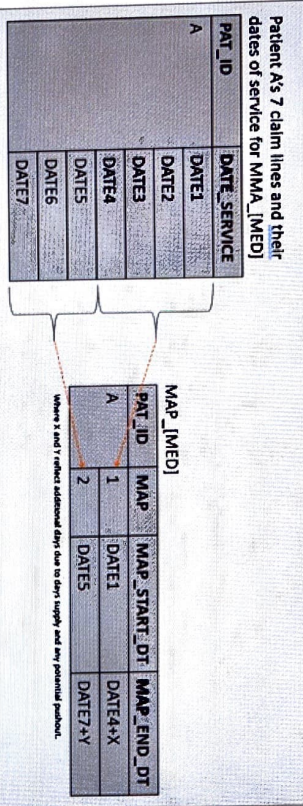


A	B	C	D	E	F	G	H
FU_PD	MAP_MED_RUNOUT_DT	The [MED] MAP's medical claim runout date		For all [MED] claims with DATE_SERVICE >= MAP_START_DT: if CLAIM_TYPE = medical then: if earliest CLAIM_TYPE=medical claim then: MAP_MED_RUNOUT_DT=DATE_SERVICE+DAY_SUPPLY-1 if not earliest CLAIM_TYPE=medical claim then: if DATE_SERVICE > max(MAP_RX_RUNOUT_DT, MAP_MED_RUNOUT_DT) then: stop processing this MAP's MAP_RX_RUNOUT_DT and MAP_MEDICAL_RUNOUT_DT else then: MAP_MED_RUNOUT_DT = DATE_SERVICE + DAY_SUPPLY-1	This variable must be defined concurrently with MAP_RX_RUNOUT_DT while processing a given MAP. For a given [MED], the MAP algorithm uses a patient's DATE_SERVICE, DAYS_SUPPLY, and CLAIM_TYPE from the MMA_[MED] dataset to create the patient's MAP(s). The MAP algorithm iterates over a MMA_[MED]'s claim/claim-line's DATE_SERVICE and applies pushout and runout rules. If a patient has concurrent pharmacy and medical claims/claim-lines, then pharmacy and medical runout dates will be separately calculated. See Figures 1-3 in Section B for more details.		
14							
FU_PD	MAP_END_DT	The end date for this [MED] MAP	date	max(MAP_RX_RUNOUT_DT, MAP_MED_RUNOUT_DT)	For a given [MED], the MAP algorithm uses a patient's DATE_SERVICE, DAYS_SUPPLY, and CLAIM_TYPE from the MMA_[MED] dataset to create the patient's MAP(s). The MAP algorithm iterates over a MMA_[MED]'s claim/claim-line's DATE_SERVICE and applies pushout and runout rules to determine a MAP_END_DT and the subsequent start of a potential new MAP. If a patient has concurrent pharmacy and medical claims/claim-lines, then pharmacy and medical runout dates will be separately calculated and used to create an eventual MAP_END_DT. See Figures 1-3 in Section B for more details.		
15							
FU_PD	MAP_CNT	The patient specific MAP order number for [MED]	numeric	=1 for a patient's first [MED] MAP, =2 for a patient's second [MED] second MAP, =3 for a patient's third [MED] MAP,	[MED] Claim level to define, multiple patient-map rows for final MAP_[MED] dataset		

Figure 1. Simplified Creation of MAP_[MED]



In this simplified example, Patient A's claims are processed one at a time and the first 4 claim/claim-line's day supply are combined to create MAP 1 because their fifth claim/claim-line occurs after the first 4 claims medications (and their day supply/pushout) have run out. Thus, MAP1 end date is set based on the day supply / pushout after combining the first 4 claim lines and the final 3 claim/claim-lines are combined to create MAP 2 (where DATE5 > DATE4+X). Note that while the MAP_START_DT equals an exact date from a claim/claim-line, the MAP_END_DT is an initially unknown value and must be calculated using each claim/claim-line's day supply.

Figure 2. Pharmacy Claim Pushout / Runout Example

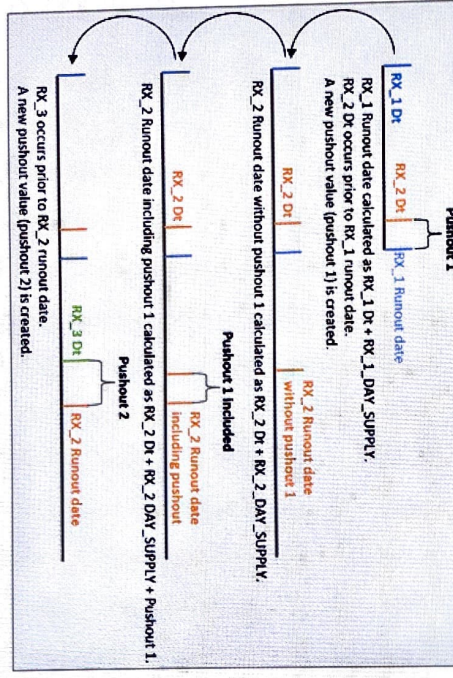


Figure 3. MAP Algorithm Logic Example

MMA_[MED] Dataset Input		MAP Algorithm Details	
		Pharmacy	Medical

Each MAP.

- Has a start and end date.
- Is at the Patient-MAP level (that is, a patient may have multiple MAPs for the same medication and a patient who received multiple MM medications will have at least one MAP for each MM medication received).
- Has nonoverlapping MAP time periods if the a patient has multiple MAPs for a given [MED].
- Is a patient-claim level dataset containing the processed claims having an [MED] MMA medication code belonging to the patients found in the patient input dataset (see INPUT DATASETS tab). For example, MMA_BORT would only contain those processed claims with an NDC or HCPCS code for BORT belonging to patients found in the PATIENT_INPUT dataset.
- A single patient will appear more than once if they have multiple claim/claim-lines indicating use of [MED].

1. Pushout - If a pharmacy claim occurs on day X and has a 28 day supply, assume patient has medication available on day X thru the claim's DATE_SERVICE + 27 days (X+28-1). If a subsequent pharmacy claim for the same medication occurs prior to X+27 on day Y, then pushout_Y is defined as (X+27 - Y) + 1 and captures the days of medication the patient still had available from their earlier fill upon receiving their next pharmacy fill. Conceptually, if a patient has an "early refill" in pharmacy claims, we assume the patient begins the "early refill" [MED] after the runout date of the previous [MED] fill.
 2. Pharmacy runout date - The date a patient no longer has medication available based on pharmacy claims. Calculated as the date of the pharmacy claim + days supply + potential pushout.
 3. Medical runout date - The date a patient no longer has medication available in their body based on medical claims. Calculated as the date of the medical claim + 28. Pushout is not implemented.
- A patient with multiple pharmacy claims, each occurring prior to the current pharmacy runout, will generate a new pushout value for each combination of adjacent claims. See Figure 2 for more details.

For a given patient in MMA_[MED], their earliest claim/claim-line's DATE_SERVICE that occurs after their index date will be the

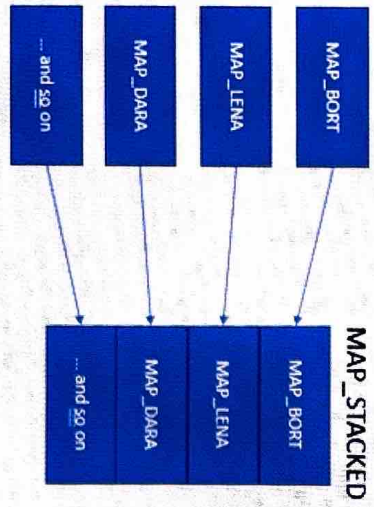
Figure 3. MAP Algorithm Logic Example

MMA_[MED] Dataset Input				MAP Algorithm Details		
PAT_ID	DATE_SERVICE	CLAIM_TYPE	DAY_SUPPLY	Iteration	Pharmacy Runout	Medical Runout
A	May 15	Pharmacy	30	1	Jun 13	
	Jun 10	Pharmacy	30	2	Jul 13	
	Jun 25	Medical	30	3	Jul 13	Jul 24
	Jul 15	Pharmacy	30	4	Aug 13	Jul 24
	Aug 10	Medical	30	5	Aug 13	Sep 8
	Sep 5	Medical	30	6	Aug 13	Oct 4
Next claim on NOV 7 occurs after both pharmacy and medical runout dates.						
	Nov 7	Pharmacy	30	7	DEC 6	

[MED] MAP Algorithm Output for PAT_ID A

PAT_ID	MAP_CNT	MAP_START_DT	MAP_END_DT	MAP_MED_TYPE	MAP_MED_CLASS
A	1	May 15	Oct 4	[MED]	[MED Class]
	2	Nov 7	DEC 6	[MED]	[MED Class]

Figure. Stacking MAP_MED datasets together to create MAP_STACKED



Each MAP_[MED] dataset (see tab MAP_MED) is at the patient-map level (a patient can have multiple MAPs for the same MED). After stacking the MAP datasets together, the resulting MAP_STACKED dataset can be thought of as at the MED-patient-map level.

For a given patient in MMA_[MED], their earliest claim/claim-line's DATE_SERVICE that occurs after their index date will be the MAP_START_DT of their first [MED] MAP. Set the pharmacy and medical runout dates based on the claim's CLAIM_TYPE.

If the chronological next claim/claim-line occurred after both the patient's pharmacy and medical runout dates, then the set the MAP_END_DT to the maximum of either the pharmacy or medical runout dates and the claim's DATE_SERVICE will be the MAP_START_DT of the patient's next MAP.

Otherwise, if the next claim occurred on or before either the pharmacy or medical runout dates, use the CLAIM_TYPE to update the appropriate runout date (pharmacy or medical), incorporating pushout when updating the pharmacy runout date. Note that it may be possible for a pharmacy claim to occur after the pharmacy runout date but before the medical runout date. This does not end the MAP, rather the pharmacy runout is updated to be the DATE_SERVICE of the pharmacy claim + DAY_SUPPLY-1 (see Figure 3, MAP Algorithm Iteration 4).

Stack (not merge) each of the MAP_[MED] datasets into one combined dataset MAP_STACKED. MAPS_STACKED will have the exact same variables as the MAP_[MED] datasets but will be comprised of each the MAP_[MED] datasets. If we were to focus on a given patient that was on BORT and LENA, then MAP_STACKED would contain all of the patient's BORT and LENA MAPs (assuming the patient was present in each of the individual MAP_BORT and MAP_LENA datasets).

A B C D E F G H

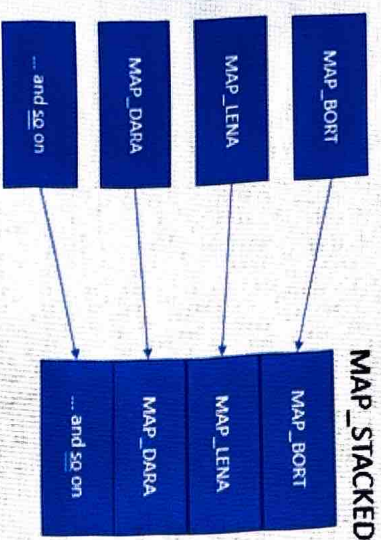


Figure. Stacking MAP_MED datasets together to create MAP_STACKED

Each MAP_[MED] dataset (see tab MAP_MED) is at the patient-map level (a patient can have multiple MAPs for the same MED). After stacking the MAP datasets together, the resulting MAP_STACKED dataset can be thought of as at the MED-patient-map level.

Stack (not merge) each of the MAP_[MED] datasets into one combined dataset MAP_STACKED. MAPS_STACKED will have the exact same variables as the MAP_[MED] datasets but will be comprised of each the MAP_[MED] datasets. If we were to focus on a given patient that was on BORT and LENA, then MAP_STACKED would contain all of the patient's BORT and LENA MAPs (assuming the patient was present in each of the individual MAP_BORT and MAP_LENA datasets).